Category C Problems - Answers

Classes and Objects Revision | Week 6 | Java solutions with verified output

M1. Student Placement Record Management

Replaces three parallel arrays with a PlacementRecord class; an array of objects keeps name, company and package together so they can never go out of sync.

Input

```
Ravi, TCS, 4.5
Anitha, Zoho, 6.2
Karthik, Infosys, 4.0
```

Solution (M1.java)

```
class PlacementRecord {
    String studentName;
    String company;
    double packageLpa;

    PlacementRecord(String studentName, String company, double packageLpa) {
        this.studentName = studentName;
        this.company = company;
        this.packageLpa = packageLpa;
    }

    void printRecord() {
        System.out.println(studentName + " -> " + company + " @ " + packageLpa + " LPA");
    }
}

public class M1 {
    public static void main(String[] args) {
        PlacementRecord[] records = {
            new PlacementRecord("Ravi", "TCS", 4.5),
            new PlacementRecord("Anitha", "Zoho", 6.2),
            new PlacementRecord("Karthik", "Infosys", 4.0)
        };

        for (PlacementRecord r : records) {
            r.printRecord();
        }
    }
}
```

Output

```
Ravi -> TCS @ 4.5 LPA
Anitha -> Zoho @ 6.2 LPA
Karthik -> Infosys @ 4.0 LPA
```

M2. Hostel Mess Wallet Management

balance is private, so it can only change through topUp() and deduct(), both of which validate input. getBalance() is read-only; there is no setter.

Input

```
opening = 500
topUp(200)
deduct(1000)
```

Solution (M2.java)

```
class MessWallet {
    private double balance;

    public MessWallet(double openingBalance) {
        if (openingBalance < 0) {
            System.out.println("Warning: negative opening balance, starting at 0");
            this.balance = 0;
        } else {
            this.balance = openingBalance;
        }
    }

    public void topUp(double amount) {
        if (amount <= 0) {
            System.out.println("Top-up rejected: amount must be positive");
            return;
        }
        balance += amount;
    }

    public void deduct(double amount) {
        if (amount > balance) {
            System.out.println("Deduct rejected: insufficient balance");
            return;
        }
        balance -= amount;
    }

    public double getBalance() {
        return balance;
    }
}

public class M2 {
    public static void main(String[] args) {
        MessWallet wallet = new MessWallet(500);
        wallet.topUp(200);
        System.out.println("Balance after top-up: " + wallet.getBalance());
        wallet.deduct(1000);
        System.out.println("Final balance: " + wallet.getBalance());
    }
}
```

Output

```
Balance after top-up: 700.0
Deduct rejected: insufficient balance
Final balance: 700.0
```

M3. Course Credit Management

The three-argument constructor uses this(code, title, credits, 0) to reuse the four-argument constructor, so the setup logic is written only once.

Input

```
21CSC201J, Data Structures, 4 (theory-only)
21CSC205L, DSA Lab, 3, labCredits 1
```

Solution (M3.java)

```
class Course {
    String code;
    String title;
    int credits;
    int labCredits;

    public Course(String code, String title, int credits, int labCredits) {
        this.code = code;
        this.title = title;
        this.credits = credits;
        this.labCredits = labCredits;
    }

    // Theory-only course: chains to the four-argument constructor
    public Course(String code, String title, int credits) {
        this(code, title, credits, 0);
    }

    public int totalCredits() {
        return credits + labCredits;
    }
}

public class M3 {
    public static void main(String[] args) {
        Course theory = new Course("21CSC201J", "Data Structures", 4);
        Course lab = new Course("21CSC205L", "DSA Lab", 3, 1);

        System.out.println(theory.code + " total credits: " + theory.totalCredits());
        System.out.println(lab.code + " total credits: " + lab.totalCredits());
    }
}
```

Output

```
21CSC201J total credits: 4
21CSC205L total credits: 4
```

M4. Library ID Card Management

Assigning one reference to another copies the reference, not the object, so both variables point to the same IdCard. == compares references, so a separate object with equal field values is still not ==.

Input

```
IdCard ravi = new IdCard("Ravi", 0);
IdCard duplicate = ravi;
duplicate.booksIssued = 3;
IdCard separate = new IdCard("Ravi", 3);
```

Solution (M4.java)

```
class IdCard {
    String name;
    int booksIssued;

    IdCard(String name, int booksIssued) {
        this.name = name;
        this.booksIssued = booksIssued;
    }
}

public class M4 {
    public static void main(String[] args) {
        IdCard ravi = new IdCard("Ravi", 0);
        IdCard duplicate = ravi;          // same object, not a new one
        duplicate.booksIssued = 3;        // change via second reference

        IdCard separate = new IdCard("Ravi", 3);  // different object, same values

        System.out.println("Ravi's booksIssued (via first variable): " + ravi.booksIssued);
        System.out.println("duplicate == ravi: " + (duplicate == ravi));
        System.out.println("separate == ravi: " + (separate == ravi));
    }
}
```

Output

```
Ravi's booksIssued (via first variable): 3
duplicate == ravi: true
separate == ravi: false
```

M5. Student and College Information Management

collegeName and studentCount are static, so they exist once per class instead of once per object. The constructor increments studentCount, and the static printCollegeInfo() touches no instance field, so it is called via Student.printCollegeInfo().

Input

```
2 Student objects created
```

Solution (M5.java)

```
class Student {
    String name;
    int attendance;

    static String collegeName = "SRM Institute of Science and Technology";
    static int studentCount = 0;

    Student(String name, int attendance) {
        this.name = name;
        this.attendance = attendance;
        studentCount++;
    }

    static void printCollegeInfo() {
        System.out.println(collegeName);
        System.out.println("Students created: " + studentCount);
    }
}

public class M5 {
    public static void main(String[] args) {
        Student s1 = new Student("Ravi", 90);
        Student s2 = new Student("Anitha", 95);

        Student.printCollegeInfo();
    }
}
```

Output

```
SRM Institute of Science and Technology
Students created: 2
```